

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №5-6
“ Шаблоны проектирования и модульное тестирование в Python”

Выполнил:
Студент группы ИУ5-34Б
Прощаев А.П.
Преподаватель:
Нардид А.Н.

Москва 2025

Постановка задачи

Необходимо для произвольной предметной области реализовать от одного до трех шаблонов проектирования: один порождающий, один структурный и один поведенческий. Для сдачи лабораторной работы в минимальном варианте достаточно реализовать один паттерн.

В модульных тестах необходимо применить следующие технологии:

1. TDD - фреймворк.
2. BDD - фреймворк.

Создание Mock-объектов.

Предметной области

В качестве предметной области выбрано управление задачами. Имеются простые, срочные и составные задачи.

Шаблоны проектирования

Порождающий шаблон: Фабричный метод (Factory method). Используется для удобного создания заданий различных типов.

Структурный шаблон: Компоновщик (Composite). Используется для реализации составных задач, состоящих из подзадач различных типов.

Поведенческий шаблон: Наблюдатель или Издатель – подписчик (Observer). Используется для осуществления механизма обновления статуса составной задачи при обновлении статусов подзадач.

Код программы

Tasks/Tasks.py

```
from abc import ABC, abstractmethod
from enum import Enum

from datetime import datetime, timedelta

class TaskType(Enum):
    URGENT = 0
    SIMPLE = 1
    COMPOSITE = 2
```

```

class Task(ABC):
    def __init__(self, description):
        self.description = description
        self.completed = False
        self.observers = []

    def add_observer(self, observer):
        if observer not in self.observers:
            self.observers.append(observer)

    def remove_observer(self, observer):
        if observer in self.observers:
            self.observers.remove(observer)

    def notify_observers(self):
        for observer in self.observers:
            observer.update_status()

    def mark_as_completed(self):
        self.completed = True
        self.notify_observers()

    def is_completed(self):
        return self.completed

    @abstractmethod
    def get_status(self):
        pass

class SimpleTask(Task):
    def __init__(self, description):
        super().__init__(description)

    def get_status(self):
        return {
            'Description': self.description,
            'Task type': TaskType.SIMPLE,
            'Completed': self.completed,
        }

class UrgentTask(Task):
    def __init__(self, description, deadline):
        super().__init__(description)
        self.deadline = deadline
        self.deadline = deadline.replace(microsecond=0)

    def get_status(self):
        delta = self.deadline - datetime.now().replace(microsecond=0)

        time_left = str(delta) if delta > timedelta(0) else 'expired'

        return {

```

```

        'Description': self.description,
        'Task type': TaskType.URGENT,
        'Completed': self.completed,
        'Time left': str(time_left)
    }

```

Tasks/TaskFactory.py

```

from Tasks.Tasks import TaskType, SimpleTask, UrgentTask
from Tasks.CompositeTask import CompositeTask

class TaskFactory:
    def create_task(self, task_type, *args, **kwargs):
        if task_type == TaskType.URGENT:
            return UrgentTask(*args, **kwargs)
        elif task_type == TaskType.SIMPLE:
            return SimpleTask(*args, **kwargs)
        elif task_type == TaskType.COMPOSITE:
            return CompositeTask(*args, **kwargs)
        else:
            raise ValueError(f'Unknown task type: {task_type}')

```

Tasks/CompositeTask.py

```

from Tasks.Tasks import Task, TaskType
from Tasks.TaskObserver import TaskObserver

class CompositeTask(Task, TaskObserver):
    def __init__(self, description):
        super().__init__(description)
        self.components = []

    def add_component(self, component):
        if component not in self.components:
            self.components.append(component)
            component.add_observer(self)

    def remove_component(self, component):
        if component in self.components:
            self.components.remove(component)
            component.remove_observer(self)

    def mark_as_completed(self):
        super().mark_as_completed()
        for component in self.components:
            component.mark_as_completed()

    def update_status(self):
        if self.is_completed():
            return

```

```

        if all(component.is_completed() for component in
self.components):
            self.mark_as_completed()

    def get_status(self):
        subtasks = [component.get_status() for component in
self.components]

        status = {
            'Description': self.description,
            'Task type': TaskType.COMPOSITE,
            'Completed': self.completed,
            'Subtasks': subtasks
        }

        return status

```

Tasks/TaskObserver.py

```

from abc import ABC, abstractmethod

class TaskObserver(ABC):
    @abstractmethod
    def update_status(self):
        pass

```

BDD фреймворк

Tests/BDD/task.feature

Feature:

As a progressive guy

I want to create simple tasks

To not forget about them

Scenario: Creation simple task Cook dinner

When I create a simple task "Cook dinner"

Then task has description: "Cook dinner"

Scenario: Mark as completed

Given the same task "Cook dinner"

When I mark task as completed

Then task gets completed

Tests/BDD/steps/task_steps.py

```
from behave import given, when, then

from Tasks.Tasks import SimpleTask

@when('I create a simple task "{description}"')
def step_impl(context, description):
    context.task = SimpleTask(description)

@then('task has description: "{correct_description}"')
def step_impl(context, correct_description):
    assert context.task.description == correct_description

@given('the same task "{description}"')
def step_impl(context, description):
    context.task = SimpleTask(description)

@when('I mark task as completed')
def step_impl(context):
    context.task.mark_as_completed()

@then('task gets completed')
def step_impl(context):
    assert context.task.is_completed() == True
```

TDD фреймворк

Tests/TDD/test_SimpleTask.py

```
import unittest

from Tasks.Tasks import SimpleTask, TaskType

class TestSimpleTask(unittest.TestCase):
    def test_create_task(self):
        task = SimpleTask('Cook dinner')

        self.assertEqual(task.description, 'Cook dinner')
        self.assertFalse(task.completed)

    def test_mark_as_completed(self):
        task = SimpleTask('Cook dinner')

        task.mark_as_completed()
        self.assertTrue(task.completed)
```

```

def test_is_completed(self):
    task = SimpleTask('Cook dinner')
    self.assertFalse(task.is_completed())

    task.mark_as_completed()
    self.assertTrue(task.is_completed())

def test_get_status(self):
    task = SimpleTask('Cook dinner')
    status = task.get_status()

    correct_status = {
        'Description': 'Cook dinner',
        'Task type': TaskType.SIMPLE,
        'Completed': False,
    }

    self.assertEqual(status, correct_status)

```

Tests/TDD/test_UrgentTask.py

```

import unittest
from unittest.mock import patch

from datetime import datetime

from Tasks.Tasks import UrgentTask, TaskType

class TestUrgentTask(unittest.TestCase):
    def test_create_task(self):
        task = UrgentTask('Do homework', datetime(2025, 12, 9, 16))

        self.assertEqual(task.description, 'Do homework')
        self.assertFalse(task.completed)
        self.assertEqual(task.deadline, datetime(2025, 12, 9, 16))

    def test_create_task_deadline_with_microsecs(self):
        task = UrgentTask('Do homework', datetime(2025, 12, 9, 16, 0,
0, 200))

        self.assertEqual(task.deadline, datetime(2025, 12, 9, 16))

    @patch('Tasks.Tasks.datetime')
    def test_get_status(self, mock_datetime):
        fixed_time = datetime(2025, 12, 9, 16, 16, 44)
        mock_datetime.now.return_value = fixed_time

        deadline = datetime(2025, 12, 9, 17)
        task = UrgentTask('Do homework', deadline)
        status = task.get_status()

        correct_status = {

```

```

        'Description': 'Do homework',
        'Task type': TaskType.URGENT,
        'Completed': False,
        'Time left': '0:43:16'
    }

    self.assertEqual(status, correct_status)

@patch('Tasks.Tasks.datetime')
def test_get_status_deadline_more_than_month(self, mock_datetime):
    fixed_time = datetime(2025, 11, 9, 16, 16, 44)
    mock_datetime.now.return_value = fixed_time

    deadline = datetime(2025, 12, 15, 16)
    task = UrgentTask('Do homework', deadline)
    status = task.get_status()

    correct_status = {
        'Description': 'Do homework',
        'Task type': TaskType.URGENT,
        'Completed': False,
        'Time left': '35 days, 23:43:16'
    }

    self.assertEqual(status, correct_status)

def test_get_status_expired_task(self):
    deadline = datetime(2020, 12, 9)
    task = UrgentTask('Do homework', deadline)
    status = task.get_status()

    correct_status = {
        'Description': 'Do homework',
        'Task type': TaskType.URGENT,
        'Completed': False,
        'Time left': 'expired'
    }

    self.assertEqual(status, correct_status)

@patch('Tasks.Tasks.datetime')
def test_get_status_expired_just_now_task(self, mock_datetime):
    fixed_time = datetime(2025, 12, 9, 16, 16, 44, 200)
    mock_datetime.now.return_value = fixed_time

    deadline = datetime(2025, 12, 9, 16, 16, 44)
    task = UrgentTask('Do homework', deadline)
    status = task.get_status()

    correct_status = {
        'Description': 'Do homework',
        'Task type': TaskType.URGENT,
        'Completed': False,

```



```

        'Time left': 'expired'
    }

    self.assertEqual(status, correct_status)

```

Tests/TDD/test_TaskFactory.py

```

import unittest

from datetime import datetime

from Tasks.Tasks import TaskType
from Tasks.TaskFactory import TaskFactory

class TestTaskFactory(unittest.TestCase):
    def setUp(self):
        self.factory = TaskFactory()

    def test_create_simple_task(self):
        simple_task = self.factory.create_task(TaskType.SIMPLE, 'Cook
dinner')

        self.assertEqual(simple_task.description, 'Cook dinner')
        self.assertFalse(simple_task.completed)

    def test_create_urgent_task(self):
        simple_task = self.factory.create_task(TaskType.URGENT, 'Do
homework', datetime(2025, 12, 9, 16))

        self.assertEqual(simple_task.description, 'Do homework')
        self.assertFalse(simple_task.completed)
        self.assertEqual(simple_task.deadline, datetime(2025, 12, 9,
16))

    def test_create_composite_task(self):
        composite_task = self.factory.create_task(TaskType.COMPOSITE,
'Cook dinner')

        self.assertEqual(composite_task.description, 'Cook dinner')
        self.assertFalse(composite_task.completed)
        self.assertFalse(composite_task.components)

    def test_raise_exception_when_create_task_(self):
        with self.assertRaises(ValueError):
            task = self.factory.create_task(1, 'Cook dinner')

```

Tests/TDD/test_CompositeTask.py

```

import unittest
from unittest.mock import patch

```

```

from datetime import datetime

from Tasks.TaskFactory import TaskFactory
from Tasks.Tasks import TaskType
from Tasks.CompositeTask import CompositeTask

class TestCompositeTask(unittest.TestCase):
    def setUp(self):
        self.factory = TaskFactory()

    def test_add_component(self):
        tasks_type = TaskType.SIMPLE
        simple_tasks = CompositeTask('Simple Tasks')

        cook_soup_task = self.factory.create_task(tasks_type, 'Cook
soup')
        cut_vegetables_task = self.factory.create_task(tasks_type,
'Cut vegetables')
        fry_meat_task = self.factory.create_task(tasks_type, 'Fry
meat')

        simple_tasks.add_component(cook_soup_task)
        simple_tasks.add_component(cut_vegetables_task)
        simple_tasks.add_component(fry_meat_task)

        correct_components = [
            cook_soup_task,
            cut_vegetables_task,
            fry_meat_task
        ]

        for i in range(len(simple_tasks.components)):
            component = simple_tasks.components[i]
            correct_component = correct_components[i]

            self.assertEqual(component.description,
correct_component.description)

    def test_observers_when_add_component(self):
        cook_dinner_task = CompositeTask('Cook dinner')
        cook_soup_task = self.factory.create_task(TaskType.SIMPLE,
'Cook soup')
        cook_dinner_task.add_component(cook_soup_task)

        self.assertEqual(cook_soup_task.observers, [cook_dinner_task])

        cut_vegetables_task =
self.factory.create_task(TaskType.SIMPLE, 'Cut vegetables')
        cook_dinner_task.add_component(cut_vegetables_task)

        self.assertEqual(cut_vegetables_task.observers,
[cook_dinner_task])

```

```

def test_notify_observers(self):
    cook_dinner_task = CompositeTask('Cook dinner')
    cook_soup_task = self.factory.create_task(TaskType.SIMPLE,
'Cook soup')
    cook_dinner_task.add_component(cook_soup_task)

    self.assertEqual(len(cook_dinner_task.components), 1)

    cook_soup_task.mark_as_completed()

    self.assertTrue(cook_soup_task.is_completed())
    self.assertTrue(cook_dinner_task.is_completed())


def test_remove_component(self):
    tasks_type = TaskType.SIMPLE
    simple_tasks = CompositeTask('Simple Tasks')

    cook_soup_task = self.factory.create_task(tasks_type, 'Cook
soup')

    simple_tasks.add_component(cook_soup_task)
    simple_tasks.remove_component(cook_soup_task)

    self.assertFalse(simple_tasks.components)


def test_get_status(self):
    cook_dinner_task = CompositeTask('Cook dinner')

    cook_dinner_task.add_component(self.factory.create_task(TaskType.SIMPLE, 'Cook soup'))

    cook_dinner_task.add_component(self.factory.create_task(TaskType.SIMPLE, 'Cut vegetables'))

    correct_status = {
        'Description': 'Cook dinner',
        'Task type': TaskType.COMPOSITE,
        'Completed': False,
        'Subtasks': [
            {
                'Description': 'Cook soup',
                'Task type': TaskType.SIMPLE,
                'Completed': False,
            },
            {
                'Description': 'Cut vegetables',
                'Task type': TaskType.SIMPLE,
                'Completed': False,
            }
        ]
    }

```

```

    ]
}

self.assertEqual(cook_dinner_task.get_status(),
correct_status)

@patch('Tasks.Tasks.datetime')
def test_get_status_complex_composite_task(self, mock_datetime):
    fixed_time = datetime(2025, 12, 9, 16, 16, 44)
    mock_datetime.now.return_value = fixed_time

    cook_dinner_task =
self.factory.create_task(TaskType.COMPOSITE, 'Cook dinner')

    deadline = datetime(2025, 12, 9, 17)

    cook_dinner_task.add_component(self.factory.create_task(TaskType.URGEN
T, 'Cook soup', deadline))

    cook_dinner_task.add_component(self.factory.create_task(TaskType.SIMPL
E, 'Cut vegetables'))

    clean_house_task = self.factory.create_task(TaskType.SIMPLE,
'Clean the house')

    home_task = CompositeTask('Home Tasks')

    home_task.add_component(cook_dinner_task)
    home_task.add_component(clean_house_task)

    correct_status = {
        'Description': 'Home Tasks',
        'Task type': TaskType.COMPOSITE,
        'Completed': False,
        'Subtasks': [
            {
                'Description': 'Cook dinner',
                'Task type': TaskType.COMPOSITE,
                'Completed': False,
                'Subtasks': [
                    {
                        'Description': 'Cook soup',
                        'Task type': TaskType.URGENT,
                        'Completed': False,
                        'Time left': '0:43:16'
                    },
                ],
            },
            {
                'Description': 'Cut vegetables',
                'Task type': TaskType.SIMPLE,
                'Completed': False,
            },
        ],
    },

```

```

        {
            'Description': 'Clean the house',
            'Task type': TaskType.SIMPLE,
            'Completed': False,
        }
    ]
}

self.assertEqual(home_task.get_status(), correct_status)

def test_mark_as_completed_composite_task(self):
    cook_dinner_task =
self.factory.create_task(TaskType.COMPOSITE, 'Cook dinner')

cook_dinner_task.add_component(self.factory.create_task(TaskType.SIMPLE, 'Cook soup'))

cook_dinner_task.add_component(self.factory.create_task(TaskType.SIMPLE, 'Cut vegetables'))

self.assertFalse(cook_dinner_task.completed)

for component in cook_dinner_task.components:
    self.assertFalse(component.completed)

cook_dinner_task.mark_as_completed()

self.assertTrue(cook_dinner_task.completed)

for component in cook_dinner_task.components:
    self.assertTrue(component.completed)

def test_mark_task_completed_when_all_subtasks_completed(self):
    cook_soup_task = self.factory.create_task(TaskType.SIMPLE, 'Cook soup')
    cut_vegetables_task =
self.factory.create_task(TaskType.SIMPLE, 'Cut vegetables')

    cook_dinner_task =
self.factory.create_task(TaskType.COMPOSITE, 'Cook dinner')
    cook_dinner_task.add_component(cook_soup_task)
    cook_dinner_task.add_component(cut_vegetables_task)

    self.assertFalse(cook_dinner_task.is_completed())

    cook_soup_task.mark_as_completed()

    self.assertFalse(cook_dinner_task.is_completed())

    cut_vegetables_task.mark_as_completed()

    self.assertTrue(cook_dinner_task.is_completed())

```

Результат работы программы

```

● (venv) proshchaevalexey@MacBook-Air-Aleksej-2 lab_5-6 % behave ./Tests/BDD/
USING RUNNER: behave.runner:Runner
Feature: # Tests/BDD/task.feature:1
  As a progressive guy
  I want to create simple tasks
  To not forget about them
  Scenario: Creation simple task Cook dinner # Tests/BDD/task.feature:6
    When I create a simple task "Cook dinner" # Tests/BDD/steps/task_steps.py:6 0.000s
    Then task has description: "Cook dinner" # Tests/BDD/steps/task_steps.py:10 0.000s

  Scenario: Mark as completed # Tests/BDD/task.feature:10
    Given the same task "Cook dinner" # Tests/BDD/steps/task_steps.py:15 0.000s
    When I mark task as completed # Tests/BDD/steps/task_steps.py:19 0.000s
    Then task gets completed # Tests/BDD/steps/task_steps.py:23 0.000s

1 feature passed, 0 failed, 0 skipped
2 scenarios passed, 0 failed, 0 skipped
5 steps passed, 0 failed, 0 skipped
Took 0min 0.000s
● (venv) proshchaevalexey@MacBook-Air-Aleksej-2 lab_5-6 % python3 -m unittest discover ./Tests/TDD/
.....
-----
Ran 22 tests in 0.001s

OK
○ (venv) proshchaevalexey@MacBook-Air-Aleksej-2 lab_5-6 %

```